Under the Hood — The Complete Hands-On Reference

ENTITY LIFECYCLE • FETCH TYPES • CASCADES • PROPAGATION
PROJECTIONS • INHERITANCE • LOCKING • JPQL vs NATIVE SQL

A practical guide with runnable code examples for every core JPA/Hibernate concept

§ Table of Contents

1. Architecture — How a Request Flows 3
2. Entity Lifecycle & Persistence Context 4
3. Dirty Checking & Flushing 5
4. Fetch Types — LAZY vs EAGER 6
5. Associations — @OneToOne, @OneToMany, @ManyToOne, @ManyToMany 7
6. Cascade Types 8
7. Inheritance Strategies 9
8. Projections (Interface, DTO, Dynamic) 10
9. Locking — Optimistic & Pessimistic 11
10. Transaction Propagation 12
11. JPQL vs Native SQL 13
12. The N+1 Query Problem & Solutions 14
13. Transaction Isolation Levels — Dirty, Non-Repeatable & Phantom Reads 15
14. ACID, @Transactional & Rollback / Exception Handling 17
15. Plain Transactions & Savepoints (JDBC / MySQL) 19
16. Spring Proxies & Open Session in View (OSIV) 21
17. Spring Application Events & Transactional Listeners 23
18. JPA Lifecycle Callbacks (@PrePersist, @PostLoad, etc.) 25
19. Spring Data Repository Hierarchy 27
20. Using Built-In Repository Methods — Internal Working 30
★ Bonus: Two-Method @Transactional Rollback Deep Dive 33
22. Key Takeaways & Best Practices 36

1 Architecture — How a Request Flows

Every JPA-backed Spring application follows the same request pipeline. Understanding each layer is essential before diving into the deep concepts, because dirty checking, flushing, and lazy loading all happen at specific points in this pipeline.

1. Controller — Receives the HTTP request and delegates to the Service layer.

2. Service Layer (@Transactional) — Contains business logic and defines the transaction boundary.

3. Repository (Spring Data JPA) — Provides an abstraction over CRUD and custom queries.

4. EntityManager — The bridge between your application code and the persistence context.

5. Persistence Context (1st-level cache) — Tracks every managed entity: lifecycle state, dirty checking, and flush timing.

6. Hibernate (ORM provider) — Translates JPA operations into SQL and manages object-relational mapping.

7. JDBC Driver — Sends the generated SQL to the database and processes the results.

8. Database — Persists data and returns query results back up the chain.

> **Key insight:** The power of JPA is not in writing queries — it's in understanding how the Persistence Context manages entity state between the Repository and the Database. Everything else in this guide (lifecycle, dirty checking, fetch types, cascades) is really about controlling what happens inside that layer.

Minimal Project Setup

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

```
# application.properties
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

2 Entity Lifecycle & Persistence Context

Every JPA entity moves through four distinct states during its lifetime. Knowing which state an entity is in tells you exactly what Hibernate will do with it on the next flush.

State	Description	Tracked by Persistence Context?
New (Transient)	Created with `new`, not yet associated with any EntityManager.	No
Managed (Persistent)	Associated with an active persistence context; any field change is auto-tracked.	Yes
Detached	Was managed, but the persistence context closed (e.g. transaction ended) or it was explicitly detached.	No
Removed	Marked for deletion; removed from the database on next flush.	Yes (until flush)

Hands-On: Moving Through Each State

```
@Entity
public class Employee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private Double salary;
    // getters/setters
}

@Service
public class EmployeeLifecycleDemo {

    @PersistenceContext
    private EntityManager em;

    @Transactional
    public void demonstrateLifecycle() {
        Employee emp = new Employee();      // 1. NEW / TRANSIENT
        emp.setName("Asha Rao");
        emp.setSalary(75000.0);

        em.persist(emp);                        // 2. MANAGED (INSERT queued)

        emp.setSalary(80000.0);                  // still MANAGED — dirty checking
                                                  // picks this up automatically

        em.detach(emp);                          // 3. DETACHED — no longer tracked
        emp.setSalary(90000.0);                  // change is now IGNORED by JPA

        Employee managedAgain = em.merge(emp);  // re-attached, salary=90000 synced

        em.remove(managedAgain);                 // 4. REMOVED — DELETE queued on flush
    }
}
```

> **Rule of thumb:** `persist()` moves NEW → MANAGED, `detach()` / `close()` / `clear()` moves MANAGED → DETACHED, `merge()` moves DETACHED → MANAGED (a new copy), and `remove()` moves MANAGED → REMOVED.

The Persistence Context (First-Level Cache)

The Persistence Context is a per-transaction, in-memory map of every managed entity, keyed by its identity. It gives you three guarantees:

- **Identity guarantee:** within one persistence context, fetching the same row twice returns the same Java object reference.
- **Automatic dirty checking:** Hibernate compares each managed entity against a snapshot taken at load time to detect changes.
- **Write-behind:** INSERT/UPDATE/DELETE statements are queued and only sent to the database at flush time — not immediately when you call the setter.

```
@Transactional
public void identityGuarantee() {
    Employee e1 = em.find(Employee.class, 1L);
    Employee e2 = em.find(Employee.class, 1L);
    System.out.println(e1 == e2); // true — same object, no 2nd SELECT fired
}
```

3 Dirty Checking & Flushing

Dirty Checking

When a transaction commits (or flushes), Hibernate compares each managed entity's current field values against the snapshot it took when the entity was loaded. Any difference is automatically converted into an UPDATE statement — **you never call `save()` for updates on a managed entity.**

```
@Transactional
public void updateSalary(Long employeeId, Double newSalary) {
    Employee emp = employeeRepository.findById(employeeId).orElseThrow();
    emp.setSalary(newSalary);
    // NOTHING else needed. On commit, Hibernate diffs the snapshot
    // vs current state and issues: UPDATE employee SET salary=? WHERE id=?
}
```

Flush — Synchronizing Context with the Database

A flush pushes the SQL for every pending change in the persistence context to the database. It does **not** commit the transaction — it just synchronizes state so that subsequent queries in the same transaction see consistent data.

FlushModeType	Behavior
`AUTO` (default)	Hibernate flushes automatically before a JPQL/Criteria query that could be affected by pending changes, and always before commit.
`COMMIT`	Only flushes at transaction commit — queries may return stale (pre-change) data within the same transaction.

```
// Manual flush — forces pending INSERT/UPDATE/DELETE to run now
em.persist(newEmployee);
em.flush();  // SQL sent now, but row is still only visible
             // inside this transaction until commit

// Common use case: get a generated ID before building a related entity
em.persist(order);
em.flush();
System.out.println(order.getId()); // populated after flush
```

> **Common pitfall:** Calling `flush()` in a loop for bulk inserts is expensive because each flush is a network round trip. For bulk operations, batch inserts using `hibernate.jdbc.batch_size` and periodically call `flush()` + `clear()` together to avoid the persistence context growing unbounded.

4 Fetch Types — LAZY vs EAGER

Fetch type controls **when** Hibernate loads an associated entity or collection relative to when the owning entity is loaded.

Fetch Type	When data loads	Default for
`FetchType.EAGER`	Immediately, in the same query (or an immediate join/second query)	`@ManyToOne` , `@OneToOne`
`FetchType.LAZY`	Deferred until the association is first accessed	`@OneToMany` , `@ManyToMany`

```
@Entity
public class Order {
    @Id @GeneratedValue
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)   // override the EAGER default
    @JoinColumn(name = "customer_id")
    private Customer customer;

    @OneToMany(mappedBy = "order", fetch = FetchType.LAZY)
    private List<OrderItem> items = new ArrayList<>();
}
```

Why LAZY loading can throw an exception

```
Order order = orderRepository.findById(1L).orElseThrow();
// ... transaction/session closes here (e.g. leaving @Transactional method) ...

order.getItems().size();  // LazyInitializationException!
                          // No open session to fetch the collection
```

Fixes for LazyInitializationException:

- Access the lazy collection *inside* the `@Transactional` method, before it returns.
- Use a **fetch join** in JPQL: `select o from Order o join fetch o.items where o.id = :id`
- Use an `@EntityGraph` to specify which associations to eagerly load per-query.
- Use DTO projections that only select the fields you actually need (see Section 8).

@EntityGraph — Per-Query Fetch Control

```
@Entity
@NamedEntityGraph(
    name = "Order.withItems",
    attributeNodes = @NamedAttributeNode("items")
)
public class Order { /* ... */ }

public interface OrderRepository extends JpaRepository<Order, Long> {

    @EntityGraph(value = "Order.withItems", type = EntityGraph.EntityGraphType.FETCH)
    List<Order> findAll();
}
```

Best practice: Default almost everything to `LAZY` , including `@ManyToOne` / `@OneToOne` . Opt into eager loading explicitly, per query, with a fetch join or entity graph. This avoids accidentally pulling entire object graphs and is the single biggest lever for controlling the N+1 problem (Section 12).

5 Associations

JPA maps four relational relationship types. The owning side (the one holding the foreign key) uses `@JoinColumn`; the inverse side uses `mappedBy`.

@ManyToOne — Many Orders belong to One Customer

```
@Entity
public class Order {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id") // owning side — FK lives here
    private Customer customer;
}
```

@OneToMany — One Customer has Many Orders (inverse side)

```
@Entity
public class Customer {
    @OneToMany(mappedBy = "customer", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Order> orders = new ArrayList<>();

    // helper method keeps both sides of the bidirectional link in sync
    public void addOrder(Order order) {
        orders.add(order);
        order.setCustomer(this);
    }
}
```

@OneToOne — One Employee has One ParkingSpot

```
@Entity
public class Employee {
    @OneToOne(mappedBy = "employee", cascade = CascadeType.ALL)
    private ParkingSpot parkingSpot;
}

@Entity
public class ParkingSpot {
    @OneToOne
    @JoinColumn(name = "employee_id") // owning side
    private Employee employee;
}
```

@ManyToMany — Students and Courses

```
@Entity
public class Student {
    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();
}

@Entity
public class Course {
    @ManyToMany(mappedBy = "courses") // inverse side
    private Set<Student> students = new HashSet<>();
}
```

Common pitfall: Never put `@ManyToMany` on both sides without `mappedBy` — Hibernate will create two separate join tables. Also avoid `toString()` / `equals()` that traverses lazy associations; it easily triggers `LazyInitializationException` or infinite recursion in bidirectional relations.

6 Cascade Types

Cascades propagate an EntityManager operation performed on a *parent* entity to its *associated* entities, so you don't have to save/remove each child manually.

CascadeType	Effect
`PERSIST`	Saving the parent also saves any new (transient) children.
`MERGE`	Merging the parent also merges detached children.
`REMOVE`	Deleting the parent also deletes its children.
`REFRESH`	Refreshing the parent from the DB also refreshes children.
`DETACH`	Detaching the parent also detaches children.
`ALL`	Shortcut for all five operations above.

```
@Entity
public class Customer {
    @OneToMany(mappedBy = "customer", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Order> orders = new ArrayList<>();
}

@Transactional
public void createCustomerWithOrders() {
    Customer customer = new Customer("Neha");
    Order order1 = new Order("Laptop");
    customer.addOrder(order1); // order1 is transient

    customerRepository.save(customer);
    // CascadeType.ALL persists `order1` automatically —
    // no need to call orderRepository.save(order1)
}
```

orphanRemoval vs CascadeType.REMOVE

`orphanRemoval = true` is stricter than `CascadeType.REMOVE`: it deletes a child the moment it's *removed from the collection*, even if the parent itself is not deleted.

```
customer.getOrders().remove(order1); // order1 row DELETED on flush
                                     // because orphanRemoval=true, even though
                                     // the customer itself was never removed
```

Common pitfall: Never set `cascade = CascadeType.ALL` on a `@ManyToOne`/owning side pointing "up" toward a shared parent (e.g. Order → Customer). That would delete the Customer whenever a single Order is deleted. Cascade should generally flow "down" the ownership hierarchy (parent → children), not up.

7 Inheritance Strategies

JPA supports mapping a Java class hierarchy onto relational tables in four ways.

Strategy	Tables	Trade-off
`SINGLE_TABLE` (default)	One table for the whole hierarchy, with a discriminator column.	Fastest reads (no joins), but many nullable columns.
`JOINED`	One table per class, joined by primary key.	Normalized, no wasted columns, but requires joins to read a subtype.
`TABLE_PER_CLASS`	One complete table per concrete class (fields duplicated).	No joins needed, but polymorphic queries require UNION and identity/PK generation is trickier.
`@MappedSuperclass`	Not an entity itself — just shares fields/mappings with subclasses.	No hierarchy table or polymorphic queries at all; pure code reuse.

SINGLE_TABLE

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "payment_type")
public abstract class Payment {
    @Id @GeneratedValue
    private Long id;
    private Double amount;
}

@Entity
@DiscriminatorValue("CREDIT_CARD")
public class CreditCardPayment extends Payment {
    private String cardNumber;
}

@Entity
@DiscriminatorValue("PAYPAL")
public class PaypalPayment extends Payment {
    private String paypalEmail;
}
// One "payment" table: id, amount, payment_type, card_number, paypal_email
```

JOINED

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Vehicle {
    @Id @GeneratedValue
    private Long id;
    private String plateNumber;
}

@Entity
public class Car extends Vehicle {
    private Integer numberOfDoors;
}
// Tables: vehicle(id, plate_number), car(id [FK to vehicle], number_of_doors)
```

@MappedSuperclass — Shared Auditing Fields

```
@MappedSuperclass
public abstract class Auditable {
    @CreatedDate    private LocalDateTime createdAt;
    @LastModifiedDate private LocalDateTime updatedAt;
}

@Entity
public class Product extends Auditable {
    @Id @GeneratedValue
    private Long id;
    private String name;
}
// Product table gets: id, name, created_at, updated_at — no "Auditable" table exists
```

8 Projections

Projections let a query return only the fields you need instead of a full entity — reducing memory use, avoiding lazy-loading traps, and often letting the database do less work.

1. Interface-Based (Closed) Projection

```
public interface EmployeeSummary {
    String getName();
    Double getSalary();
}

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<EmployeeSummary> findBySalaryGreaterThan(Double salary);
}
// Hibernate generates: SELECT e.name, e.salary FROM employee e WHERE e.salary > ?
// — only these two columns are fetched, not the whole row
```

2. Interface-Based (Open) Projection with SpEL

```
public interface EmployeeNameView {
    @Value("#{target.name + ' (' + target.department.name + ')'}")
    String getDisplayName();
}
// Open projections evaluate a SpEL expression AFTER loading full data,
// so they lose the "fetch only these columns" optimization of closed projections.
```

3. Class-Based (DTO) Projection

```
public class EmployeeDto {
    private final String name;
    private final Double salary;

    public EmployeeDto(String name, Double salary) { // constructor matches JPQL "new" expr
        this.name = name;
        this.salary = salary;
    }
    // getters
}

@Query("SELECT new com.example.dto.EmployeeDto(e.name, e.salary) FROM Employee e")
List<EmployeeDto> findAllSummaries();
```

4. Dynamic Projection

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    <T> List<T> findByDepartment(String department, Class<T> type);
}

// Caller decides the shape at call time:
List<EmployeeSummary> summaries = repo.findByDepartment("Engineering", EmployeeSummary.class);
List<Employee> fullEntities   = repo.findByDepartment("Engineering", Employee.class);
```

> **When to use which:** Prefer **closed interface projections** for simple flat reads (best query-generation optimization). Prefer **DTO projections** when you need to combine/transform fields, use constructors, or return nested/complex shapes. Avoid **open projections** in hot paths since they defeat column-level fetch optimization.

9 Locking — Optimistic & Pessimistic

Locking controls how concurrent transactions touching the same row are handled to prevent lost updates.

Optimistic Locking — @Version

Assumes conflicts are rare. Each entity carries a version column; Hibernate checks it in the `WHERE` clause of the UPDATE and throws if another transaction changed it first.

```
@Entity
public class Account {
    @Id @GeneratedValue
    private Long id;
    private BigDecimal balance;

    @Version
    private Long version; // auto-incremented by Hibernate on every UPDATE
}

// Generated SQL: UPDATE account SET balance=?, version=? WHERE id=? AND version=?
// If 0 rows affected -> OptimisticLockException is thrown

try {
    Account acc = accountRepository.findById(1L).orElseThrow();
    acc.setBalance(acc.getBalance().subtract(BigDecimal.TEN));
    accountRepository.save(acc);
} catch (ObjectOptimisticLockingFailureException ex) {
    // someone else updated this row first — retry or notify the user
}
```

Pessimistic Locking — Database-Level Row Locks

Assumes conflicts are likely, so it locks the row immediately, blocking other transactions until this one commits.

```
public interface AccountRepository extends JpaRepository<Account, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT a FROM Account a WHERE a.id = :id")
    Optional<Account> findByIdForUpdate(@Param("id") Long id);
}
// Generated SQL: SELECT ... FROM account WHERE id=? FOR UPDATE
```

LockModeType	Meaning
`PESSIMISTIC_READ`	Shared lock — blocks other writers, allows other readers.
`PESSIMISTIC_WRITE`	Exclusive lock — blocks other readers and writers (`SELECT ... FOR UPDATE`).
`PESSIMISTIC_FORCE_INCREMENT`	Exclusive lock that also forces the version column to increment.
`OPTIMISTIC`	Verifies the version hasn't changed at transaction commit, without locking rows.
`OPTIMISTIC_FORCE_INCREMENT`	Same as OPTIMISTIC but always bumps the version, even without a field change.

> **When to choose which:** Use **optimistic locking** for high-read, low-conflict workloads (typical web CRUD) — better throughput. Use **pessimistic locking** for high-contention critical sections like bank transfers or seat reservations, where retry storms from optimistic failures would be worse than the cost of blocking.

10 Transaction Propagation

Propagation defines how a `@Transactional` method behaves when it is called from *within* another transaction.

Propagation	Behavior
`REQUIRED` (default)	Joins the existing transaction, or creates a new one if none exists.
`REQUIRES_NEW`	Always starts a brand-new transaction, suspending any existing one.
`NESTED`	Runs within a savepoint of the existing transaction; a rollback here rolls back only to the savepoint.
`MANDATORY`	Must run inside an existing transaction — throws if none exists.
`SUPPORTS`	Joins a transaction if one exists; otherwise runs non-transactionally.
`NOT_SUPPORTED`	Suspends any existing transaction and runs non-transactionally.
`NEVER`	Must run without a transaction — throws if one exists.

REQUIRED vs REQUIRES_NEW — Independent Audit Logging

```
@Service
public class OrderService {

    @Autowired private AuditService auditService;

    @Transactional // Propagation.REQUIRED by default
    public void placeOrder(Order order) {
        orderRepository.save(order);
        auditService.log("Order placed: " + order.getId());
        if (order.getTotal() > 100000) {
            throw new RuntimeException("Suspicious order amount");
            // this rolls back BOTH orderRepository.save AND the audit log,
            // because auditService.log runs in the SAME transaction (REQUIRED)
        }
    }
}

@Service
public class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void log(String message) {
        auditRepository.save(new AuditEntry(message));
        // runs in its OWN transaction, commits independently —
        // the audit entry survives even if placeOrder() later rolls back
    }
}
```

NESTED — Partial Rollback with Savepoints

```
@Transactional
public void processItems(List<Item> items) {
    for (Item item : items) {
        try {
            itemProcessor.process(item); // Propagation.NESTED
        } catch (ItemException ex) {
            // rolls back only this item's savepoint;
            // previously processed items in this loop remain committed
            log.warn("Skipping bad item: {}", item.getId());
        }
    }
}
```

Common pitfall: Propagation only takes effect through a Spring proxy — calling an `@Transactional` method from *another method in the same class* bypasses the proxy entirely, so the propagation setting is silently ignored. Move the method to a separate `@Service` bean to make propagation actually apply.

11 JPQL vs Native SQL

	JPQL	Native SQL
Operates on	Entity objects & their fields	Actual database tables & columns
Portability	Database-independent	Tied to a specific database dialect
Returns	Managed entities (in persistence context) by default	Raw rows unless mapped explicitly
Best for	Standard CRUD-style querying, entity graphs	Vendor-specific functions, complex joins, performance tuning, CTEs, window functions

JPQL Examples

```
@Query("SELECT e FROM Employee e WHERE e.department.name = :dept AND e.salary > :min")
List<Employee> findHighEarners(@Param("dept") String dept, @Param("min") Double min);

// Fetch join to avoid N+1 (loads Employee + Department in ONE query)
@Query("SELECT e FROM Employee e JOIN FETCH e.department WHERE e.id = :id")
Optional<Employee> findWithDepartment(@Param("id") Long id);

// Aggregate functions work the same as SQL
@Query("SELECT d.name, AVG(e.salary) FROM Employee e JOIN e.department d GROUP BY d.name")
List<Object[]> avgSalaryByDepartment();
```

Native SQL Examples

```
// nativeQuery = true switches to raw SQL against actual table/column names
@Query(value = "SELECT * FROM employee WHERE salary > :min ORDER BY salary DESC LIMIT 10",
       nativeQuery = true)
List<Employee> findTopEarners(@Param("min") Double min);

// Database-specific window function — not expressible in portable JPQL
@Query(value = """
    SELECT *, RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) as rnk
    FROM employee
    """, nativeQuery = true)
List<Object[]> rankEmployeesByDepartment();
```

Named/Derived Queries (No JPQL/SQL needed)

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    // Spring Data derives the JPQL from the method name itself
    List<Employee> findByDepartmentNameAndSalaryGreaterThan(String deptName, Double salary);
}
```

> **Guideline:** Default to JPQL / derived queries for portability and entity-graph awareness. Drop to native SQL only when you need a vendor-specific feature (CTEs, window functions, JSON functions, full-text search) or need to hand-tune a query for performance after profiling — not as a first resort.

12 The N+1 Query Problem & Solutions

The N+1 problem happens when you run 1 query to fetch N parent rows, then Hibernate silently runs N additional queries — one per row — to lazily fetch each row's association.

```
// 1 query to get all departments...
List<Department> departments = departmentRepository.findAll();

for (Department d : departments) {
    // ...then ONE extra query PER department to lazily load its employees
    System.out.println(d.getEmployees().size()); // N additional SELECTs!
}
```

Solution 1 — Fetch Join

```
@Query("SELECT DISTINCT d FROM Department d JOIN FETCH d.employees")
List<Department> findAllWithEmployees(); // single SQL JOIN — 1 query total
```

Solution 2 — @EntityGraph

```
@EntityGraph(attributePaths = {"employees"})
List<Department> findAll(); // overrides the repository's default findAll()
```

Solution 3 — Batch Fetching

```
@Entity
public class Department {
    @OneToMany(mappedBy = "department")
    @BatchSize(size = 20) // groups lazy loads into batches of 20 IDs
    private List<Employee> employees;
}
// Instead of N queries, Hibernate issues ceil(N/20) queries using
// "WHERE department_id IN (?, ?, ?, ... up to 20 ids)"
```

> **Detecting it:** Turn on `spring.jpa.show-sql=true` plus `hibernate.generate_statistics=true` in a test environment, or use a tool like p6spy/datasource-proxy, to spot the repeated SELECT pattern before it reaches production.

13 Transaction Isolation Levels

Isolation level controls how much one transaction is allowed to "see" of another transaction's uncommitted or concurrent changes. The SQL standard defines four levels, which prevent (or allow) three specific read anomalies. Understanding the anomalies first makes the levels make sense.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
`READ_UNCOMMITTED`	Possible	Possible	Possible
`READ_COMMITTED`	Prevented	Possible	Possible
`REPEATABLE_READ`	Prevented	Prevented	Possible
`SERIALIZABLE`	Prevented	Prevented	Prevented

The Three Anomalies, Explained

- **Dirty Read:** Transaction A reads a row that Transaction B has changed but not yet committed. If B rolls back, A has read data that never really existed.
- **Non-Repeatable Read:** Transaction A reads the same row twice in one transaction and gets two different values, because Transaction B updated and committed that row in between A's two reads.
- **Phantom Read:** Transaction A runs the same range query twice and gets a different *set of rows* the second time, because Transaction B inserted or deleted rows matching that range in between.

Setting the Isolation Level in Spring

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void transferFunds(Long fromId, Long toId, BigDecimal amount) {
    // business logic
}
```

```
// Or set a JDBC-level default for the whole datasource:
// spring.datasource.hikari.transaction-isolation=TRANSACTION_REPEATABLE_READ
```

Hands-On 1: Dirty Read — READ_UNCOMMITTED

Two threads/transactions touching the same `Account` row. Transaction B updates the balance but has not committed; Transaction A, running at `READ_UNCOMMITTED`, reads it anyway.

```
// --- Transaction B (uncommitted) ---
@Transactional(isolation = Isolation.READ_UNCOMMITTED)
public void debitWithoutCommitting() {
    Account acc = em.find(Account.class, 1L);
    acc.setBalance(acc.getBalance().subtract(BigDecimal.valueOf(500)));
    // simulate slow processing before commit
    Thread.sleep(5000);
    throw new RuntimeException("Rollback! Something failed downstream");
}

// --- Transaction A, running concurrently ---
@Transactional(isolation = Isolation.READ_UNCOMMITTED)
public BigDecimal readBalanceDuringB() {
    Account acc = em.find(Account.class, 1L);
    return acc.getBalance();
    // DIRTY READ: sees the -500 balance even though B is about to roll back.
    // After B rolls back, this returned value never actually existed in the DB.
}
```

> **Note:** Most production databases (PostgreSQL, Oracle) do not actually implement `READ_UNCOMMITTED` — they silently upgrade it to `READ_COMMITTED`. MySQL/InnoDB is one of the few that honors it literally, so this demo is easiest to reproduce there.

Hands-On 2: Non-Repeatable Read — READ_COMMITTED

```
// --- Transaction A ---
@Transactional(isolation = Isolation.READ_COMMITTED)
public void readTwice() {
```

```
    Account first = em.find(Account.class, 1L);
    System.out.println("First read: " + first.getBalance()); // e.g. 1000

    // ... meanwhile, Transaction B runs and COMMITS a change to the same row ...
    // UPDATE account SET balance = 1500 WHERE id = 1; COMMIT;

    em.refresh(first); // force a fresh read from the DB within the same transaction
    System.out.println("Second read: " + first.getBalance()); // 1500 — NON-REPEATABLE READ
}
```

What changed vs. READ_UNCOMMITTED: Transaction A never saw B's change while it was *uncommitted* (no dirty read). But once B committed, A's next read within the same transaction saw a different value than its first read — that's the non-repeatable read, and `READ_COMMITTED` does not prevent it.

Hands-On 3: Phantom Read — Fixed by REPEATABLE_READ (mostly)

```
// --- Transaction A ---
@Transactional(isolation = Isolation.REPEATABLE_READ)
public void countHighEarnersTwice() {
    Long firstCount = countEmployeesOver("100000");
    System.out.println("First count: " + firstCount); // e.g. 5

    // ... meanwhile, Transaction B INSERTs a new Employee with salary=120000 and COMMITs ...

    Long secondCount = countEmployeesOver("100000");
    System.out.println("Second count: " + secondCount);
    // On strict standard REPEATABLE_READ this COULD show 6 (a "phantom" new row)
    // because REPEATABLE_READ only guarantees rows you've ALREADY read won't change —
    // it does not fully block new rows matching your WHERE clause from appearing.
    // (PostgreSQL's REPEATABLE_READ uses snapshot isolation and actually prevents this;
    // this is dialect-dependent — verify against your specific database.)
}

private Long countEmployeesOver(String min) {
    return ((Number) em.createQuery(
        "SELECT COUNT(e) FROM Employee e WHERE e.salary > :min")
        .setParameter("min", new BigDecimal(min))
        .getSingleResult()).longValue();
}
```

Hands-On 4: Fully Preventing Phantoms — SERIALIZABLE

```
@Transactional(isolation = Isolation.SERIALIZABLE)
public void strictSeatBooking(Long eventId, Integer seatNumber) {
    Long booked = em.createQuery(
        "SELECT COUNT(b) FROM Booking b WHERE b.eventId = :eid AND b.seatNumber = :seat", Long.class)
        .setParameter("eid", eventId)
        .setParameter("seat", seatNumber)
        .getSingleResult();

    if (booked > 0) {
        throw new IllegalStateException("Seat already booked");
    }
    em.persist(new Booking(eventId, seatNumber));
    // Under SERIALIZABLE, the database guarantees this transaction behaves as if
    // it ran completely alone. If a concurrent transaction tries to book the same
    // seat at the same time, ONE of the two transactions will be forced to abort
    // with a serialization failure — you must catch and retry it.
}
```

```
// Retrying on serialization failure (PostgreSQL throws SQLState 40001)
int attempts = 0;
while (true) {
    try {
        bookingService.strictSeatBooking(eventId, seatNumber);
        break;
    } catch (CannotAcquireLockException | PessimisticLockingFailureException ex) {
        if (++attempts >= 3) throw ex;
        // brief backoff, then retry the whole transaction
    }
}
```

Trade-off: `SERIALIZABLE` gives the strongest correctness guarantees but the worst throughput — transactions abort and must be retried under contention. Reserve it for genuinely correctness-critical operations (seat/ticket booking, inventory decrement, financial

> ledgers) rather than applying it application-wide.

Quick Decision Guide

Level	Use when
`READ_UNCOMMITTED`	Almost never in production; only for approximate analytics where speed matters more than correctness.
`READ_COMMITTED`	The sensible default for most CRUD applications (and the default in PostgreSQL/Oracle/SQL Server).
`REPEATABLE_READ`	Reports or multi-step reads within one transaction that must stay internally consistent (MySQL/InnoDB's default).
`SERIALIZABLE`	Correctness-critical read-then-write logic where a phantom or race condition would cause real damage (bookings, ledgers, inventory).

14 ACID, @Transactional & Rollback / Exception Handling

ACID Properties

Property	Guarantee	Who enforces it
Atomicity	All statements in a transaction succeed, or none do — a failure rolls everything back.	Transaction manager (Spring) + DB transaction log
Consistency	A transaction moves the database from one valid state to another, never violating constraints (FKs, unique keys, checks).	Database constraints + application invariants
Isolation	Concurrent transactions don't see each other's uncommitted changes, to the degree defined by the isolation level (Section 13).	Database's MVCC / locking engine
Durability	Once committed, data survives crashes/power loss.	Database's write-ahead log (WAL) / redo log

> **Where these show up in JPA:** Atomicity and Consistency are what `@Transactional` gives you at the Spring layer; Isolation is what you configured in Section 13; Durability is entirely the database engine's job once COMMIT is issued — Hibernate has no control over it.

@Transactional — What It Actually Does

Spring wraps the annotated bean in a dynamic proxy (JDK or CGLIB). Calling the method actually calls the proxy first, which: begins a transaction (or joins one, per propagation), invokes your real method, and then commits or rolls back based on whether an exception escaped the method.

```
@Service
public class PaymentService {

    @Transactional(
        propagation = Propagation.REQUIRED,
        isolation   = Isolation.READ_COMMITTED,
        timeout     = 10,             // seconds before forced rollback
        readOnly    = false,
        rollbackFor = {PaymentException.class},
        noRollbackFor = {DuplicateNotificationException.class}
    )
    public void processPayment(Payment payment) {
        // ...
    }
}
```

> **Proxy pitfall (recap from Section 10):** Self-invocation — calling an `@Transactional` method from another method in the *same class* — bypasses the proxy, so no transaction is started/joined at all. Also, `@Transactional` only works on `public` methods by default; Spring silently ignores it on private/protected methods.

Default Rollback Behavior: Checked vs Unchecked Exceptions

This is the single most misunderstood rule in Spring transactions:

Exception type	Examples	Default rollback behavior
Unchecked (extends `RuntimeException` or `Error`)	`NullPointerException`, `IllegalStateException`, custom `RuntimeException` subclasses	**Rolls back automatically** — no configuration needed
Checked (extends `Exception` but not `RuntimeException`)	`IOException`, `SQLException`, custom checked `Exception` subclasses	**Does NOT roll back by default** — the transaction commits anyway!

```
// UNCHECKED — rolls back automatically, no config needed
@Transactional
public void withdraw(Long accountId, BigDecimal amount) {
    Account acc = accountRepository.findById(accountId).orElseThrow();
    if (acc.getBalance().compareTo(amount) < 0) {
        throw new InsufficientFundsException("Not enough balance");
        // InsufficientFundsException extends RuntimeException
        // -> Spring rolls back automatically. No @Transactional attribute needed.
    }
    acc.setBalance(acc.getBalance().subtract(amount));
}
```

```
// CHECKED — surprising default: the transaction COMMITS despite the exception!
@Transactional
public void archiveInvoice(Long invoiceId) throws InvoiceArchiveException {
    Invoice invoice = invoiceRepository.findById(invoiceId).orElseThrow();
    invoice.setStatus("ARCHIVED");  // this UPDATE is queued

    boolean uploaded = archiveStorage.upload(invoice);
    if (!uploaded) {
        throw new InvoiceArchiveException("Storage upload failed");
        // InvoiceArchiveException extends Exception (checked), NOT RuntimeException.
        // DEFAULT BEHAVIOR: Spring still COMMITS the transaction!
        // The invoice is marked ARCHIVED in the DB even though the upload failed.
    }
}
```

Forcing a Checked Exception to Roll Back — rollbackFor

```
@Transactional(rollbackFor = InvoiceArchiveException.class)
public void archiveInvoice(Long invoiceId) throws InvoiceArchiveException {
    Invoice invoice = invoiceRepository.findById(invoiceId).orElseThrow();
    invoice.setStatus("ARCHIVED");

    boolean uploaded = archiveStorage.upload(invoice);
    if (!uploaded) {
        throw new InvoiceArchiveException("Storage upload failed");
        // Now Spring rolls back — the ARCHIVED status update is undone,
        // keeping the DB state consistent with the failed upload.
    }
}
```

Preventing Rollback for an Unchecked Exception — noRollbackFor

```
@Transactional(noRollbackFor = DuplicateNotificationException.class)
public void notifyCustomer(Order order) {
    orderRepository.updateNotifiedFlag(order.getId(), true); // should persist regardless

    if (notificationService.wasAlreadySent(order)) {
        throw new DuplicateNotificationException("Already notified");
        // DuplicateNotificationException extends RuntimeException, which would
        // normally roll back — but noRollbackFor overrides that so the
        // updateNotifiedFlag() UPDATE is still committed.
    }
}
```

Attribute	Effect
`rollbackFor`	Add exception types (typically checked) that SHOULD trigger rollback beyond the default.
`noRollbackFor`	Exclude exception types (typically unchecked) that should NOT trigger rollback, despite being RuntimeExceptions.
`rollbackForClassName` / `noRollbackForClassName`	Same as above, but matched by class name string — useful when the exception class isn't compile-time visible.

Programmatic Rollback (Without Throwing)

Sometimes you want to force a rollback based on business logic without throwing an exception at all:

```
@Transactional
public void conditionalRollback(Order order) {
    orderRepository.save(order);

    if (fraudDetectionService.looksSuspicious(order)) {
        TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();
        // marks the transaction rollback-only. It will still roll back at commit
        // time even though no exception was thrown — but the method returns normally,
        // so the caller must check some other signal (return value, flag) to react.
        return;
    }
}
```

Partial Rollback — Combining Propagation with Try/Catch

A full transaction rollback undoes *everything* in that transaction. To roll back only *part* of a larger unit of work while keeping the rest committed, combine `REQUIRES_NEW` or `NESTED` (Section 10) with a try/catch around the inner call.

```
@Service
public class BulkImportService {

    @Autowired private RecordProcessor recordProcessor;

    @Transactional // outer transaction — REQUIRED by default
    public ImportResult importRecords(List<RawRecord> records) {
        int succeeded = 0, failed = 0;

        for (RawRecord raw : records) {
            try {
                recordProcessor.processOne(raw); // Propagation.NESTED — own savepoint
                succeeded++;
            } catch (RecordValidationException ex) {
                failed++;
                // The catch happens HERE, in the outer transaction, so the outer
                // transaction itself is NOT marked rollback-only. Only the savepoint
                // for this one record is rolled back — previously succeeded records
                // remain intact when the outer transaction commits.
                log.warn("Skipping invalid record {}: {}", raw.getId(), ex.getMessage());
            }
        }
        return new ImportResult(succeeded, failed);
    }
}

@Service
public class RecordProcessor {
    @Transactional(propagation = Propagation.NESTED)
    public void processOne(RawRecord raw) {
        Entity entity = mapper.toEntity(raw);
        validator.validate(entity); // throws RecordValidationException if invalid
        repository.save(entity);
    }
}
```

Why REQUIRES_NEW is NOT the same as NESTED here: `REQUIRES_NEW` commits the inner unit as a fully separate, independent transaction — useful for audit logs that must survive an outer rollback (Section 10), but it means the inner commit happens even before the outer transaction finishes, which is wrong for "try one record, keep going" use cases. `NESTED` uses a savepoint inside the *same* physical transaction and connection, so it only truly commits when the outer transaction commits — giving true "skip this one, keep the rest" partial rollback. `NESTED` requires a JDBC driver/database that supports savepoints.

The one rule that ties this whole section together: a rollback only happens if the exception *propagates out of the proxied method*. Catching an exception inside the transactional method (as in the import loop above) means Spring never sees it, so it never rolls back anything at that level — this is exactly how partial rollback is achieved on purpose, and exactly how accidental "silent success on failure" bugs happen when it's not intentional.

15 Plain Transactions & Savepoints (JDBC / MySQL)

Before trusting Spring's `@Transactional`, it helps to see what a transaction actually is underneath — plain JDBC, or even the MySQL console, with no framework involved at all.

What Is a Transaction? — MySQL Console

```
-- A transaction groups statements so they succeed or fail as one unit
START TRANSACTION;

UPDATE account SET balance = balance - 200 WHERE id = 1;
UPDATE account SET balance = balance + 200 WHERE id = 2;

-- Nothing is durable yet — another session querying `account` right now
-- still sees the OLD balances (assuming READ_COMMITTED or stricter)

COMMIT;   -- now both changes are durable and visible to everyone
-- ROLLBACK; would undo both updates instead
```

Setting Up a "Balance Enforcer" Trigger (Consistency, enforced by the DB itself)

```
DELIMITER $$
CREATE TRIGGER balance_enforcer
BEFORE UPDATE ON account
FOR EACH ROW
BEGIN
    IF NEW.balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Balance cannot go negative';
    END IF;
END$$
DELIMITER ;

-- Now even if application code has a bug, the DATABASE itself refuses
-- to let a transaction commit a negative balance — this is the "C" in
-- ACID enforced at the lowest possible layer, independent of Hibernate.
```

Transaction Savepoints — Partial Rollback at the SQL Level

```
START TRANSACTION;

UPDATE account SET balance = balance - 100 WHERE id = 1;
SAVEPOINT after_debit;

UPDATE account SET balance = balance + 100 WHERE id = 999; -- id doesn't exist!
-- 0 rows affected, but MySQL does NOT auto-rollback the whole transaction

ROLLBACK TO SAVEPOINT after_debit; -- undoes nothing here since the 2nd UPDATE
                                   -- matched no rows, but demonstrates the mechanism:
                                   -- only statements AFTER the savepoint are undone
COMMIT; -- the debit from account 1 is still committed
```

This is exactly the SQL-level mechanism that Spring's `Propagation.NESTED` (Section 10 and Section 14) uses under the hood — a JPA/Hibernate "nested transaction" is really just a JDBC savepoint.

Transaction Management Using Plain Java (No Spring, No JPA)

```
Connection conn = dataSource.getConnection();
try {
    conn.setAutoCommit(false); // starts a transaction — JDBC auto-commits by default!

    try (PreparedStatement debit = conn.prepareStatement(
            "UPDATE account SET balance = balance - ? WHERE id = ?")) {
        debit.setBigDecimal(1, amount);
        debit.setLong(2, fromId);
        debit.executeUpdate();
    }

    try (PreparedStatement credit = conn.prepareStatement(
            "UPDATE account SET balance = balance + ? WHERE id = ?")) {
```

```
        credit.setBigDecimal(1, amount);
        credit.setLong(2, toId);
        credit.executeUpdate();
    }

    conn.commit();  // both updates become durable together
} catch (SQLException ex) {
    conn.rollback(); // undoes BOTH updates — this is manual atomicity
    throw new RuntimeException("Transfer failed, rolled back", ex);
} finally {
    conn.setAutoCommit(true);
    conn.close();
}
```

> **Why this matters:** `@Transactional` is not magic — it's Spring automating exactly the `setAutoCommit(false)` / `commit()` / `rollback()` dance shown above, around a proxy, so you don't have to write it by hand in every service method.

16 Spring Proxies & Open Session in View (OSIV)

How the Spring Proxy Actually Wraps Your Bean

When you annotate a method with `@Transactional`, Spring doesn't modify your class — it creates a proxy object that sits in front of it. Every call from outside goes through the proxy first.

Proxy type	Used when	Consequence
JDK Dynamic Proxy	Your class implements at least one interface	Proxy implements the same interface(s); only interface methods are proxied
CGLIB Proxy	Your class has no interface (default in Spring Boot)	Proxy subclasses your class; `final` classes/methods cannot be proxied

```
@Service
public class OrderService {

    @Transactional
    public void placeOrder(Order order) {
        validate(order); // plain internal call — bypasses the proxy entirely
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void validate(Order order) {
        // Calling this FROM placeOrder() above does NOT start a new transaction —
        // `this.validate(order)` is a plain Java call, never routed through the proxy.
        // Fix: inject OrderService (or a separate ValidationService) and call
        // the method through that injected reference instead of `this`.
    }
}
```

Open Session in View (OSIV)

OSIV keeps the Hibernate session (and its JDBC connection) open for the entire HTTP request — not just for the duration of the `@Transactional` service method. Spring Boot enables this by default (`spring.jpa.open-in-view=true`).

```
# application.properties
spring.jpa.open-in-view=false  # most teams should explicitly turn this off
```

	OSIV enabled (default)	OSIV disabled
Lazy loading in the Controller/view layer	Works — session is still open	`LazyInitializationException` if accessed outside the transaction
DB connection held	For the whole request (even during template rendering, remote calls, etc.)	Only for the duration of the actual transaction
Connection pool pressure	Higher — connections tied up longer under load	Lower — connections released as soon as the transaction ends
Where N+1 queries surface	Silently, possibly in the view/serialization layer	Immediately and loudly, in the service layer, right where it's fixable

> **Common pitfall this masks:** With OSIV on, a controller can call `order.getItems()` after the service method returns and it "just works" — silently issuing extra SQL during JSON serialization. This hides N+1 queries (Section 12) until load testing or production reveals the real cost. Most experienced teams disable OSIV and fetch everything needed explicitly inside the transactional service method.

No-Session & LazyInitializationException — Where They Actually Come From

```
@RestController
public class OrderController {

    @GetMapping("/orders/{id}")
    public OrderDto getOrder(@PathVariable Long id) {
        Order order = orderService.findOrder(id); // transaction already closed by here
        return new OrderDto(order.getId(), order.getItems().size());
        // With OSIV off: "could not initialize proxy - no Session" (LazyInitializationException)
        // With OSIV on: works, but issues a hidden SELECT right here, in the controller
    }
}
```

> **The real fix, either way:** don't rely on OSIV to paper over the problem. Fetch what you need inside the transactional service method (fetch join, entity graph, or a DTO projection per Sections 4, 8 and 12) and return a fully-populated DTO from the service layer, so the controller never touches a lazy association at all.

17 Spring Application Events & Transactional Listeners

Spring's event system lets one part of your application publish something happened, without knowing (or caring) who reacts to it — decoupling side effects like sending an email or updating a cache from the core business transaction.

Publishing and Listening

```
public class OrderPlacedEvent {
    private final Long orderId;
    public OrderPlacedEvent(Long orderId) { this.orderId = orderId; }
    public Long getOrderId() { return orderId; }
}

@Service
public class OrderService {
    @Autowired private ApplicationEventPublisher publisher;

    @Transactional
    public void placeOrder(Order order) {
        orderRepository.save(order);
        publisher.publishEvent(new OrderPlacedEvent(order.getId()));
        // by default, listeners run SYNCHRONOUSLY, right here, still inside
        // this transaction — before it has committed
    }
}

@Component
public class OrderEventListener {
    @EventListener
    public void onOrderPlaced(OrderPlacedEvent event) {
        emailService.sendConfirmation(event.getOrderId());
    }
}
```

Conditional Event Handling

```
@EventListener(condition = "#event.orderId > 1000")
public void onlyForLargeOrderIds(OrderPlacedEvent event) {
    // SpEL condition — the listener body only runs if this evaluates true
    auditService.flagForReview(event.getOrderId());
}
```

Exception Propagation Between Listeners

```
@EventListener
public void firstListener(OrderPlacedEvent event) {
    throw new RuntimeException("Email service down");
    // By default, an exception here propagates back up to publishEvent(),
    // which means it can roll back the SAME transaction that published the
    // event — a failing listener can undo the order that was just placed!
}
```

> **Why this catches people off guard:** Synchronous `@EventListener` methods run inside the publisher's transaction and thread. An unrelated side-effect (like a flaky email service) throwing an exception can roll back completely unrelated business logic. This is the exact motivation for `@TransactionalEventListener`, next.

@TransactionalEventListener — Tying Listeners to Transaction Phases

```
@Component
public class OrderEmailListener {

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void sendConfirmationEmail(OrderPlacedEvent event) {
        // Only runs if/when the ORIGINAL transaction actually commits successfully.
        // If placeOrder() rolls back, this listener never runs at all —
        // so customers are never emailed about an order that doesn't exist.
        emailService.sendConfirmation(event.getOrderId());
    }
```

```
    @TransactionalEventListener(phase = TransactionPhase.AFTER_ROLLBACK)
    public void onOrderRolledBack(OrderPlacedEvent event) {
        metricsService.incrementFailedOrders(); // "recovering from failure"
    }

    @TransactionalEventListener(phase = TransactionPhase.BEFORE_COMMIT)
    public void validateBeforeCommit(OrderPlacedEvent event) {
        // runs just before commit — an exception HERE can still trigger a rollback,
        // unlike AFTER_COMMIT which runs only once success is already locked in
    }
}
```

TransactionPhase	Runs	Can still cause rollback?
`BEFORE_COMMIT`	Just before the transaction commits	Yes
`AFTER_COMMIT` (default)	Only after a successful commit	No — too late, already committed
`AFTER_ROLLBACK`	Only if the transaction rolled back	No
`AFTER_COMPLETION`	Always, whether commit or rollback	No

"Listeners Gone Quiet" — No Active Transaction

```
public void importFromBatchJob(Order order) { // NOT @Transactional!
    orderRepository.save(order);
    publisher.publishEvent(new OrderPlacedEvent(order.getId()));
    // @TransactionalEventListener methods are SILENTLY SKIPPED here —
    // there is no transaction to tie AFTER_COMMIT/AFTER_ROLLBACK to,
    // and Spring does not warn you; the listener just never fires.
}
```

Fallback Execution Pattern

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT, fallbackExecution = true)
public void sendConfirmationEmail(OrderPlacedEvent event) {
    // fallbackExecution=true means: if there is NO active transaction at all
    // (e.g. called from a non-transactional batch job above), run immediately
    // instead of being silently skipped. Without this flag, the "listeners
    // gone quiet" problem above happens with no error and no log line.
    emailService.sendConfirmation(event.getOrderId());
}
```

Rule of thumb: Use plain `@EventListener` for logic that must happen synchronously as part of the same transaction (and is allowed to roll it back). Use `@TransactionalEventListener(phase = AFTER_COMMIT)` for side effects that should only ever happen once the underlying data is truly durable — emails, external API calls, cache invalidation — and add `fallbackExecution = true` if that same listener might also fire from a code path with no active transaction.

18 JPA Lifecycle Callbacks

Section 2 covered the four *states* an entity moves through (New, Managed, Detached, Removed). JPA also lets you hook **callback methods** directly onto the transitions between those states — useful for auditing, validation, computed fields, or cache invalidation, without cluttering your service layer.

The Seven Callback Annotations

Annotation	Fires
`@PrePersist`	Just before a new entity's INSERT is issued (state: New → Managed)
`@PostPersist`	Just after the INSERT has been executed (id is populated by this point)
`@PreUpdate`	Just before an UPDATE is issued for a dirty managed entity
`@PostUpdate`	Just after the UPDATE has been executed
`@PreRemove`	Just before a DELETE is issued (state: Managed → Removed)
`@PostRemove`	Just after the DELETE has been executed
`@PostLoad`	Just after an entity has been loaded from the database (or refreshed)

Hands-On: Callbacks Directly on the Entity

```
@Entity
public class Employee {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private BigDecimal salary;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    @Transient // not persisted — just a runtime-only computed field
    private String salaryBand;

    @PrePersist
    public void onPrePersist() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = this.createdAt;
    }

    @PreUpdate
    public void onPreUpdate() {
        this.updatedAt = LocalDateTime.now();
    }

    @PostLoad
    public void onPostLoad() {
        // recompute a transient field every time this entity is loaded from the DB
        this.salaryBand = salary.compareTo(BigDecimal.valueOf(100000)) > 0 ? "SENIOR" : "STANDARD";
    }

    @PreRemove
    public void onPreRemove() {
        log.info("About to delete employee {}", id);
    }
}
```

Hands-On: Callbacks in a Separate @EntityListeners Class

For cross-cutting concerns (auditing multiple entity types), pull the callback methods out into a dedicated listener class instead of repeating them on every entity.

```
public class AuditListener {

    @PrePersist
    public void beforeInsert(Object entity) {
        if (entity instanceof Auditable auditable) {
            auditable.setCreatedAt(LocalDateTime.now());
        }
    }
```

```
    @PreUpdate
    public void beforeUpdate(Object entity) {
        if (entity instanceof Auditable auditable) {
            auditable.setUpdatedAt(LocalDateTime.now());
        }
    }

    @PostPersist
    public void afterInsert(Object entity) {
        log.info("Inserted: {}", entity.getClass().getSimpleName());
    }
}

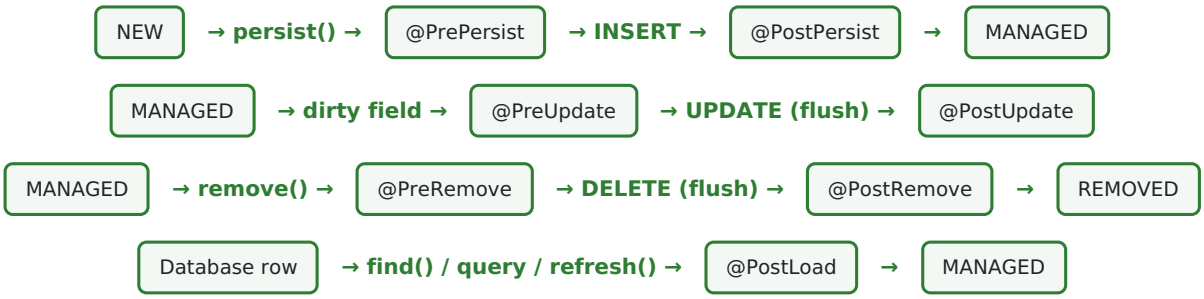
@Entity
@EntityListeners(AuditListener.class) // attach the listener to this entity
public class Invoice implements Auditable {
    @Id @GeneratedValue
    private Long id;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    // getters/setters implementing Auditable
}
```

Spring Data's Built-In Auditing (Same Idea, Zero Boilerplate)

```
@EnableJpaAuditing // on a @Configuration class
public class JpaConfig { }

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class) // Spring Data's built-in listener
public abstract class Auditable {
    @CreatedDate    private LocalDateTime createdAt;
    @LastModifiedDate private LocalDateTime updatedAt;
    @CreatedBy      private String createdBy;   // needs an AuditorAware<String> bean
    @LastModifiedBy private String updatedBy;
}
// Any entity extending Auditable gets createdAt/updatedAt/createdBy/updatedBy
// populated automatically on @PrePersist / @PreUpdate — no manual callback code at all.
```

The Full Picture: States + Callbacks Together



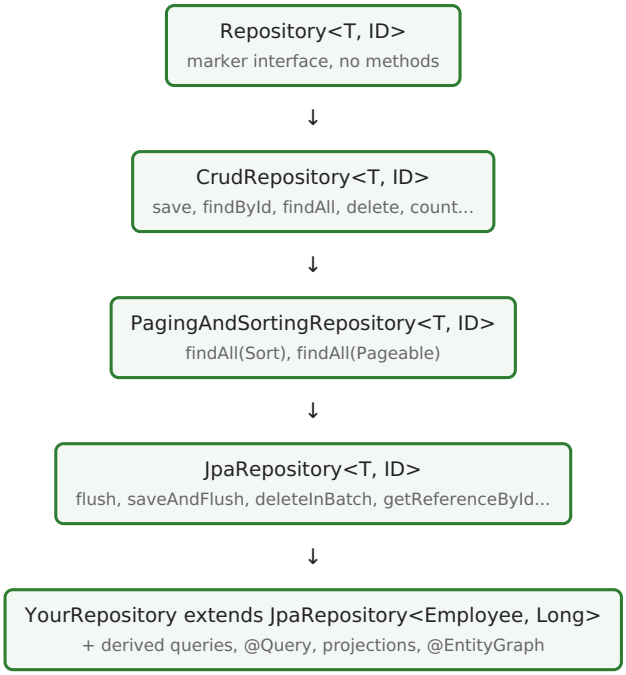
Common pitfalls:

- A callback method must return `void` and take no arguments (entity-level) or a single `Object` parameter (listener-class-level) — anything else is silently never invoked or throws at startup depending on the provider.
- `@PreUpdate` only fires when Hibernate's dirty-checking detects a changed field (Section 3) — it will not fire just because you called `save()` on an entity with no actual field changes.
- Throwing an exception from `@PrePersist` / `@PreUpdate` / `@PreRemove` aborts the operation and typically rolls back the transaction — a legitimate way to enforce invariants, but easy to trigger by accident with unrelated validation logic.
- Don't perform new `EntityManager` operations (like persisting a different entity) inside a callback — behavior here is provider-specific and can lead to subtle bugs; keep callbacks limited to mutating the current entity's own state.

19 Spring Data Repository Hierarchy

Spring Data JPA gives you CRUD, paging, sorting, and query-by-example "for free" by layering a stack of interfaces. You typically only ever write one line — `extends JpaRepository<T, ID>` — but knowing what each layer beneath it actually contributes helps you pick the leanest interface for the job, and understand exactly which methods are available and why.

The Hierarchy, Top to Bottom



Interface	Adds	Definition
`Repository<T, ID>`	Nothing — pure marker	Lets Spring Data discover the interface as a repository during component scanning; declares no methods of its own.
`CrudRepository<T, ID>`	Basic CRUD	`save`, `saveAll`, `findById`, `existsById`, `findAll`, `count`, `deleteById`, `delete`, `deleteAll`.
`PagingAndSortingRepository<T, ID>`	Paging & sorting	Extends `CrudRepository` and adds `findAll(Sort)` and `findAll(Pageable)` for ordered/paged reads.
`JpaRepository<T, ID>`	JPA-specific extras	Extends `PagingAndSortingRepository` and `QueryByExampleExecutor`; adds `flush()`, `saveAndFlush()`, batch deletes, and `getReferenceById()` (a lazy proxy without hitting the DB).
`QueryByExampleExecutor<T>`	Query by Example	Adds `findAll(Example<T>)` — build a query by populating a sample entity instance instead of writing JPQL.

Hands-On 1: Repository — The Marker Interface

```
// Repository itself declares NOTHING. This compiles and Spring Data
// still recognizes it as a repository bean, but you get zero built-in methods —
// only whatever you declare yourself.
public interface EmployeeRepository extends Repository<Employee, Long> {
    List<Employee> findByDepartmentName(String departmentName); // derived query still works
}
```

Hands-On 2: CrudRepository — Basic CRUD

```
public interface EmployeeRepository extends CrudRepository<Employee, Long> { }

@Service
public class EmployeeService {
    @Autowired private EmployeeRepository repo;

    public void demo() {
        Employee saved = repo.save(new Employee("Meera", BigDecimal.valueOf(85000)));
        Optional<Employee> found = repo.findById(saved.getId());
        boolean exists = repo.existsById(saved.getId());
        long total = repo.count();
```

```
        repo.deleteById(saved.getId());
        // CrudRepository is enough when you never need paging, sorting, or JPA extras —
        // e.g. a lightweight lookup table or reference-data repository.
    }
}
```

Hands-On 3: PagingAndSortingRepository — Paged, Sorted Reads

```
public interface EmployeeRepository extends PagingAndSortingRepository<Employee, Long>,
                                            CrudRepository<Employee, Long> { }
// (as of Spring Data 3.x, PagingAndSortingRepository no longer extends CrudRepository
// automatically — declare both, or just extend JpaRepository which includes everything.)

public Page<Employee> getEmployeesPage(int page, int size) {
    Pageable pageable = PageRequest.of(page, size, Sort.by("salary").descending());
    return repo.findAll(pageable);
    // Page includes content, totalElements, totalPages, hasNext, etc.
    // Generated SQL: SELECT ... ORDER BY salary DESC LIMIT ? OFFSET ?
    // plus a second COUNT(*) query to populate totalElements.
}
```

Hands-On 4: JpaRepository — Flush, Batch Delete, getReferenceById

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> { }

public void demo() {
    Employee emp = repo.saveAndFlush(new Employee("Kabir", BigDecimal.valueOf(72000)));
    // saveAndFlush() forces the INSERT to hit the DB immediately (Section 3),
    // instead of waiting for the transaction to commit

    List<Employee> toRemove = repo.findByDepartmentName("Contractors");
    repo.deleteAllInBatch(toRemove);
    // deleteAllInBatch() issues ONE bulk DELETE statement, vs deleteAll()
    // which loads + deletes entities one by one (and fires @PreRemove per row)

    Employee ref = repo.getReferenceById(42L);
    // Returns a lazy proxy WITHOUT hitting the DB at all yet — only useful
    // when you just need to set a foreign key reference, e.g.:
    order.setEmployee(ref); // no SELECT fired just to set this association
}
```

Hands-On 5: QueryByExampleExecutor — Query by Example

```
public List<Employee> searchByExample(String namePrefix, String department) {
    Employee probe = new Employee();
    probe.setName(namePrefix);
    probe.setDepartment(new Department(department));

    ExampleMatcher matcher = ExampleMatcher.matching()
        .withMatcher("name", ExampleMatcher.GenericPropertyMatchers.startsWith())
        .withIgnoreNullValues();

    Example<Employee> example = Example.of(probe, matcher);
    return repo.findAll(example);
    // Builds a WHERE clause purely from the non-null fields on `probe`,
    // no JPQL/native SQL written at all — handy for flexible search forms.
}
```

Your Own Repository: The Layer You Actually Write

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Derived query (Spring Data parses the method name into JPQL)
    List<Employee> findByDepartmentNameAndSalaryGreaterThan(String dept, BigDecimal min);

    // Custom JPQL with a fetch join (Section 11 & 12)
    @Query("SELECT e FROM Employee e JOIN FETCH e.department WHERE e.id = :id")
    Optional<Employee> findWithDepartment(@Param("id") Long id);

    // Projection (Section 8)
    List<EmployeeSummary> findBySalaryGreaterThan(BigDecimal salary);
```

```
    // Locking (Section 9)
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT e FROM Employee e WHERE e.id = :id")
    Optional<Employee> findByIdForUpdate(@Param("id") Long id);
}
```

Picking the right level: use plain `Repository` only if you want full control and zero inherited methods (rare); `CrudRepository` for simple lookup/reference tables with no paging needs; and `JpaRepository` — by far the most common choice — whenever you might need paging, batch operations, or JPA-specific extras like `saveAndFlush` or `getReferenceById`. There's no real runtime cost to extending the "biggest" interface, so most teams default to `JpaRepository` everywhere.

The Repository "Lifecycle" — How the Interface Becomes a Bean

1. **Component scan:** `@EnableJpaRepositories` (auto-configured by Spring Boot) scans for interfaces extending `Repository`.
2. **Proxy creation:** for each interface found, Spring Data creates a dynamic proxy at startup — you never write an implementation class.
3. **Method resolution:** when a method is called, Spring Data first checks if it's a custom/derived query, a `@Query` annotation, or a base CRUD method, and routes it to the appropriate backing implementation (`SimpleJpaRepository` for the built-ins).
4. **Execution:** `SimpleJpaRepository` ultimately delegates to the `EntityManager` — the same one from Section 1's request flow — so every repository call still goes through the persistence context, dirty checking, and flush rules covered throughout this guide.

20 Using Built-In Repository Methods — Internal Working

You call `employeeRepository.save(employee)` in a Service class and it "just works" — but a lot happens between that one line and the row landing in the database. This section traces the full call chain for the four methods you'll use every day: `save`, `findById`, `findAll`, and `delete`.

Hands-On: A Typical Service Class

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository; // interface only — you never wrote an impl

    @Transactional
    public Employee create(Employee employee) {
        return employeeRepository.save(employee);
    }

    @Transactional(readOnly = true)
    public Employee getById(Long id) {
        return employeeRepository.findById(id)
            .orElseThrow(() -> new EmployeeNotFoundException(id));
    }

    @Transactional(readOnly = true)
    public List<Employee> getAll() {
        return employeeRepository.findAll();
    }

    @Transactional
    public void remove(Long id) {
        employeeRepository.deleteById(id);
    }
}
```

What Actually Happens: `save(employee)`

1. **Proxy intercepts the call.** `employeeRepository` is not your interface directly — it's a Spring Data proxy created at startup (Section 19's repository lifecycle).
2. **Transaction proxy joins in.** Because the calling Service method is `@Transactional`, the transactional advice (Section 14/16) begins or joins a transaction *before* the repository proxy's own logic runs.
3. **Delegates to `SimpleJpaRepository`.** Spring Data's default implementation checks: does the entity have a null/default ID? If so, it calls `entityManager.persist(employee)`; otherwise `entityManager.merge(employee)`.
4. **Entity becomes Managed.** The entity is now tracked in the Persistence Context (Section 2) — its state transitions from New/Detached to Managed.
5. **SQL is queued, not sent yet.** Hibernate generates the INSERT/UPDATE statement but holds it until flush time (Section 3) — normally at transaction commit, unless something forces an earlier flush.
6. **Commit triggers flush.** The outer `@Transactional` boundary commits, Hibernate flushes the queued SQL through the JDBC driver (Section 1), and the database persists the row.

What Actually Happens: `findById(id)`

1. **Persistence Context check first.** Before touching the database at all, Hibernate checks if an entity with that ID is already Managed in the current persistence context (the identity guarantee from Section 2). If found, it's returned immediately — **zero SQL executed.**
2. **Cache miss → SELECT issued.** If not already managed, `SimpleJpaRepository` calls `entityManager.find(Employee.class, id)`, which Hibernate translates into a `SELECT ... WHERE id = ?`.
3. **Result wrapped in Optional.** Spring Data JPA wraps the (possibly null) result in `Optional<Employee>` for you — the raw JPA API returns `null` directly.
4. **Entity becomes Managed** and is now tracked for dirty checking for the rest of this transaction.

What Actually Happens: `findAll()`

1. **Always hits the database.** Unlike `findById`, there's no "check the persistence context first" shortcut for a full-table scan — Spring Data builds and runs `SELECT e FROM Employee e` (JPQL) every time.
2. **Each row becomes/updates a Managed entity.** As Hibernate processes the result set, for each row it either creates a new Managed

entity or returns the existing Managed instance if that row's ID is already in the persistence context (identity guarantee again).

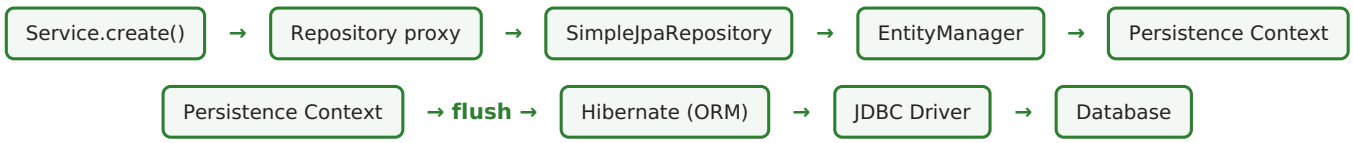
3. **Associations follow FetchType rules.** Any `@ManyToOne` / `@OneToOne` fields load eagerly by default (Section 4) unless you overrode that — this is exactly where an unnoticed N+1 (Section 12) usually starts.

What Actually Happens: `deleteById(id)`

1. **Loads the entity first.** Somewhat surprisingly, `SimpleJpaRepository.deleteById()` calls `findById()` internally before deleting — JPA's `remove()` operation requires a Managed entity, not just an ID, so it can run `@PreRemove` / `@PostRemove` callbacks (Section 18) and cascade to associations (Section 6).
2. **Marks the entity Removed.** `entityManager.remove(entity)` transitions the entity to the Removed state (Section 2).
3. **DELETE queued, then flushed at commit** — same write-behind behavior as `save()`.

> **Why this matters in practice:** Because `deleteById` loads the full entity first, deleting 10,000 rows one-by-one via a loop calling `deleteById` issues 10,000 SELECTs *plus* 10,000 DELETEs. Use `deleteAllInBatch()` (Section 19) or a bulk `@Modifying @Query("DELETE FROM Employee e WHERE ...")` for bulk operations — both skip the per-row load and lifecycle callbacks entirely, trading callback/cascade correctness for raw throughput.

Tracing It End-to-End



> **The thread that ties this whole guide together:** every single built-in repository method — no matter how simple it looks in your Service class — flows through the exact same pipeline from Section 1, obeys the exact lifecycle states from Section 2, respects the exact fetch/cascade/lock rules from Sections 4, 6 and 9, and only actually touches the database according to the flush rules from Section 3. There is no separate "Spring Data magic" layer — it's the same JPA mechanics, just with boilerplate written for you once.

★ Bonus: Two-Method @Transactional Rollback Deep Dive

This bonus section works through one recurring scenario in full: a Service class with **two `@Transactional` methods**, where the second method contains business logic that can fail. It covers self-invocation, cross-bean calls, every propagation type, and the checked-vs-unchecked rollback default — the exact combination that trips people up in interviews and in production.

The Core Rule (Applies in Every Scenario Below)

Exception type	Default rollback behavior
Unchecked — extends `RuntimeException` (or `Error`)	**Rolls back automatically.** No configuration needed.
Checked — extends `Exception`, not `RuntimeException`	**Does NOT roll back.** The transaction commits anyway unless you set `rollbackFor`.

> **Why this default exists:** Spring inherited the convention from EJB. Checked exceptions were meant to represent *expected, recoverable* business outcomes the caller should handle — so the assumption is you probably want to keep whatever succeeded. Unchecked exceptions were meant to represent *unexpected* failures — so the safe default is to undo everything.

Scenario 1 — Self-Invocation (Same Class, Plain "this." Call)

```
@Service
public class OrderService {

    @Transactional
    public void method1(Order order) {
        saveInitialState(order);
        method2(order); // plain "this." call — BYPASSES the proxy entirely
    }

    @Transactional
    public void method2(Order order) {
        if (!businessRuleValid(order)) {
            throw new RuntimeException("Business rule failed");
        }
    }
}
```

> **What actually happens:** `method2`'s own `@Transactional` is completely ignored — self-invocation never goes through the proxy (Section 16). It just runs as a plain Java call inside `method1`'s already-active transaction. The `RuntimeException` still escapes `method1` itself though, so Spring's unchecked-exception default kicks in there: the whole transaction (including `saveInitialState`) rolls back — it works, but not for the reason it looks like.

Scenario 2 — Cross-Bean Call, Default Propagation (REQUIRED)

```
@Service
public class OrderService {
    @Autowired private ValidationService validationService;

    @Transactional
    public void method1(Order order) {
        saveInitialState(order);
        validationService.method2(order); // goes through a REAL proxy this time
    }
}

@Service
public class ValidationService {
    @Transactional // REQUIRED by default — joins method1's transaction
    public void method2(Order order) {
        if (!businessRuleValid(order)) {
            throw new RuntimeException("Business rule failed");
        }
    }
}
```

> **What actually happens:** `method2`'s proxy intercepts the exception, marks the shared transaction rollback-only, and rethrows. Same end result as Scenario 1 — full rollback — but if `method1` caught the exception and tried to proceed to commit anyway, Spring throws `UnexpectedRollbackException` rather than silently committing, because the transaction was already flagged rollback-only by method2.

Scenario 3 — Cross-Bean Call, REQUIRES_NEW

```java
@Service
public class ValidationService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void method2(Order order) {
        if (!businessRuleValid(order)) {
            throw new RuntimeException("Business rule failed");
        }
    }
}

// In method1 — the difference is whether the exception is caught:
@Transactional
public void method1(Order order) {
    saveInitialState(order);
    try {
        validationService.method2(order);
    } catch (RuntimeException ex) {
        log.warn("Validation failed, continuing anyway: {}", ex.getMessage());
    }
    updateFinalState(order);
}
```

What actually happens: `REQUIRES_NEW` suspends transaction A (method1's) and opens a fully separate physical transaction B (method2's). B rolls back on its own the instant the exception fires. If `method1` doesn't catch the exception, it still escapes and transaction A rolls back too — same as Scenario 2. But if `method1` *catches* it (as shown above), transaction A commits normally: `saveInitialState` and `updateFinalState` persist, and only B's work is undone. This is the one scenario where the two methods' outcomes are genuinely independent.

Scenario 4 — Cross-Bean Call, NESTED

```java
@Transactional(propagation = Propagation.NESTED)
public void method2(Order order) {
    if (!businessRuleValid(order)) {
        throw new RuntimeException("Business rule failed");
    }
}
```

What actually happens: `NESTED` uses a JDBC **savepoint** inside the *same physical connection/transaction* as method1 — unlike `REQUIRES_NEW`, it is not a separate transaction. If `method1` catches the exception, only the savepoint (method2's work) rolls back, while everything before it survives, and the whole thing still commits together as one unit at the end. If uncaught, the entire outer transaction rolls back. Requires a JDBC driver/database that supports savepoints.

Summary Table — Propagation × Caught-or-Not

method2's propagation	Uncaught RuntimeException in method1	Caught in method1 (try/catch)
Self-invocation (no real proxy)	Whole transaction rolls back (via method1's own boundary)	Rolls back only if method1 itself doesn't swallow it
`REQUIRED` (default)	Whole transaction (A+B as one) rolls back	Spring throws `UnexpectedRollbackException` on commit
`REQUIRES_NEW`	B rolls back independently; A also rolls back (exception still propagates)	B rolled back, A commits fine — truly independent
`NESTED`	Whole transaction rolls back	Only the savepoint (B's work) rolls back; A commits

Now Layer In the Checked Exception Twist

The exception-type default is **orthogonal to propagation** — propagation decides *whose* transaction rolls back and how independently; checked-vs-unchecked decides *whether a rollback happens at all*. Swap the unchecked exception for a checked one in any scenario above, and the default flips:

```java
// ---------- UNCHECKED — rolls back automatically ----------
@Transactional
public void processOrderUnchecked(Order order) {
    orderRepository.save(order);          // INSERT queued
    if (!businessRuleValid(order)) {
        throw new IllegalStateException("Business rule failed");
        // extends RuntimeException -> automatic rollback. save() above is UNDONE.
    }
}
```

```java
// ---------- CHECKED — commits despite the exception ----------
@Transactional
public void processOrderChecked(Order order) throws BusinessRuleException {
    orderRepository.save(order);          // INSERT queued
    if (!businessRuleValid(order)) {
        throw new BusinessRuleException("Business rule failed");
        // extends Exception, NOT RuntimeException -> DEFAULT: Spring still COMMITS.
        // The save() above IS PERSISTED even though validation failed right after it.
    }
}

// ---------- THE FIX ----------
@Transactional(rollbackFor = BusinessRuleException.class)
public void processOrderChecked(Order order) throws BusinessRuleException {
    orderRepository.save(order);
    if (!businessRuleValid(order)) {
        throw new BusinessRuleException("Business rule failed");
        // Now Spring rolls back — save() is undone, consistent with the failure.
    }
}
```

This applies no matter which propagation method2 uses in Scenarios 2–4 above — swap `RuntimeException` for a checked type in any of them, add `rollbackFor`, and the rollback behavior described in each scenario still holds; without it, that scenario's method2 transaction commits instead of rolling back.

> **One-line summary:** propagation decides *which transaction* rolls back and how independently (self-invocation ignores it, `REQUIRED` shares one boundary, `REQUIRES_NEW` truly isolates, `NESTED` uses a savepoint); the checked/unchecked distinction decides *whether a rollback happens at all* — and that second rule is completely orthogonal to propagation, so always double-check it separately for every checked exception in a transactional method.

22 Key Takeaways & Best Practices

- **Spring Data JPA simplifies data access, but JPA + Hibernate do the heavy lifting** underneath — knowing the mechanics (lifecycle, dirty checking, flush) is what separates "it works" from "it scales".
- **Understand the internals** to write more efficient, correct, and maintainable applications: what state an entity is in tells you exactly what will hit the database and when.
- **Default associations to LAZY** and opt into eager loading per query with fetch joins or entity graphs — this is the single biggest lever against N+1 queries.
- **Cascade carefully and only "downward"** from parent to owned children; use `orphanRemoval` for true parent-owns-child collections.
- **Pick locking strategy by contention**: optimistic (`@Version`) for typical CRUD, pessimistic (`SELECT ... FOR UPDATE`) for high-contention critical sections.
- **Understand propagation boundaries**: `REQUIRED` shares a transaction (all-or-nothing), `REQUIRES_NEW` isolates side effects like audit logs, `NESTED` allows partial rollback.
- **Know your default rollback rules**: unchecked exceptions roll back automatically; checked exceptions do NOT unless you add `rollbackFor` — this single default catches more people off guard than almost anything else in Spring transactions.
- **Pick isolation level by workload**: `READ_COMMITTED` for typical CRUD, `REPEATABLE_READ` / `SERIALIZABLE` only where phantom or non-repeatable reads would cause real damage — stronger isolation always costs throughput.
- **Prefer JPQL/derived queries** for portability; drop to native SQL only for vendor-specific features after profiling shows a real need.
- **Use projections** (interface or DTO) whenever you don't need a full managed entity — smaller result sets, fewer lazy-loading traps, often faster queries.
- **Turn off Open Session in View** (`spring.jpa.open-in-view=false`) so lazy-loading mistakes surface immediately in the service layer instead of hiding in the controller or view.
- **Use** `@TransactionalEventListener(phase = AFTER_COMMIT)` for side effects like emails or external calls that must only happen once data is truly durable — and remember listeners are silently skipped with no active transaction unless `fallbackExecution = true`.
- **Use lifecycle callbacks** (`@PrePersist`, `@PreUpdate`, `@PostLoad`, etc.) or Spring Data's `@EnableJpaAuditing` for auditing and computed fields — but keep them side-effect-free on the current entity only; don't perform unrelated EntityManager operations inside them.
- **Default to** `JpaRepository` for new repositories — it's a superset of `CrudRepository` and `PagingAndSortingRepository` with no real runtime cost, and gives you paging, batch deletes, and `getReferenceById` if you need them later.
- **Every built-in repository call goes through the same pipeline** as everything else in this guide — proxy, `SimpleJpaRepository`, `EntityManager`, persistence context, then Hibernate/JDBC at flush time. Knowing this makes debugging "why did this fire an extra query" or "why didn't this callback run" straightforward instead of mysterious.
- **Good knowledge of JPA = better performance + fewer bugs.** Go beyond CRUD: master the concepts in this guide to build exceptional, scalable backend systems.

> **Closing thought:** The power of JPA is not just in writing queries, but in understanding how it manages your data — from the moment an entity is created in memory to the moment its SQL hits the database and the response flows back to the client.